

CodeKids

Volumen II: Stack Tecnológico y Arquitectura de Software

Proyecto: Plataforma Educativa de Programación para Niños

Equipo: Alan / Aquino / Francisco / Alonso

Periodo: Enero - Agosto 2026

1. Stack Tecnológico

La selección tecnológica de CodeKids prioriza la velocidad de ejecución, la reducción de dependencias pesadas y la integración fluida con servicios serverless.

1.1 Tecnologías de Frontend (Cliente)

Tecnología	Versión	Justificación Arquitectónica
JavaScript Vanilla	ES6+	Se evitó el uso de frameworks masivos (React/Angular) para garantizar tiempos de carga ultrarrápidos y mantener el control absoluto sobre el DOM y los eventos de red.
Tailwind CSS	v3	Proporciona un flujo de diseño basado en utilidades, permitiendo crear el complejo layout de tres columnas (estilo MS Teams) sin escribir archivos CSS gigantes y difíciles de mantener.
Font Awesome	v6.5.1	Librería de iconografía vectorial para toda la interfaz, vital para un diseño visualmente atractivo para los niños.
Google Fonts	N/A	Tipografías Fredoka (títulos), Poppins (interfaz) y Nunito (textos), elegidas por su legibilidad y estética amigable.
Google Blockly	v11.2.1	El motor central del laboratorio. Transforma conceptos complejos de programación en bloques visuales encajables, compilando silenciosamente el resultado a código ejecutable.
Leaflet	v1.9.4	Renderizado del mapa de escuelas en la página pública sin los altos costos asociados a la API de Google Maps.
Chart.js	v4.4.0	Generación de gráficos estadísticos en los dashboards de progreso y rendimiento.
SweetAlert2	N/A	Reemplaza las alertas nativas del navegador por cuadros de diálogo modales interactivos y estéticamente acordes al diseño gamificado.
Fetch API	Nativa	Manejo de peticiones asíncronas hacia el backend proxy sin requerir librerías externas como Axios.

1.2 Backend y Servicios en la Nube

Servicio Cloud	Función Específica
Firebase Auth	Motor de autenticación para validar credenciales, manejar sesiones persistentes y generar tokens JWT.
Cloud Firestore	Base de datos NoSQL de documentos en tiempo real. Sincroniza datos instantáneamente en los clientes (vital para el módulo de chat).
Firebase Storage	Almacenamiento de objetos binarios (BLOBs) como imágenes de avatares, capturas de tareas entregadas y archivos adjuntos de lecciones.
Firebase Hosting	Distribución de archivos estáticos (HTML/CSS/JS) a través de una red global de entrega de contenido (CDN) segura con certificados SSL automáticos.
Firebase Functions	Capa de lógica de negocio serverless (Node.js). Protege operaciones sensibles, como el proxy hacia la Inteligencia Artificial y la auditoría.
Render.com	Proveedor de infraestructura de respaldo configurado vía <code>render.yaml</code> para alta disponibilidad.

2. Estructura de Directorios del Repositorio

El código fuente de CodeKids sigue un patrón de diseño limpio y separaciones lógicas de responsabilidades. La raíz del proyecto `CodeKids-Clean/` se estructura de la siguiente manera:

```
CodeKids-Clean/
|-- index.html           # Redireccionador de entrada (hacia
Pagina_Inicio)
|-- firebase.json        # Configuración de Firebase y emuladores
|-- firestore.rules      # Políticas de seguridad declarativas de la BD
|-- firestore.indexes.json # Índices compuestos de consultas
|-- storage.rules        # Políticas de acceso a archivos multimedia
|-- render.yaml          # Definición de infraestructura en Render.com
|-- package.json         # Dependencias y scripts de Node.js
|
|-- Vistas_Publicas/     # Módulo sin autenticación
|   |-- Pagina_Inicio.html # Landing page, mapa Leaflet
|   |-- Inicio_De_Sesion.html # Portal central de login
|   |-- Beneficios_Profesor.html # Documentación de marketing
|
|-- auth/
|   |-- register.html     # Creación de cuentas (Ruta protegida para Admin)
|
|-- Dashboard/           # Interfaces protegidas por autenticación
|   |-- Dashboard_Administrador/ # Consola de gestión y auditoría
|   |-- Dashboard_Estudiante/    # Entorno lúdico, lecciones, lab Blockly
|   |-- Dashboard_Profesor/      # Panel de seguimiento, grupos y calificaciones
|
|-- js/                  # Controladores JavaScript (Lógica de negocio)
|   |-- firebase-config.js # Instanciación de SDKs
|   |-- auth.js           # Manejo de tokens y cambio de contraseñas
|   |-- app.js            # Core de la aplicación
|   |-- gamification.js   # Algoritmos de XP, niveles y medallas
|   |-- laboratorio.js    # Integración de Google Blockly
|   |-- admin.js          # Control de usuarios y entidades
|   |-- chat-enhancements.js # Motor de websockets/tiempo real para mensajes
|   |-- state-manager.js  # Control de estado global
|
|-- css/                 # Archivos de estilos e importaciones Tailwind
|-- images/              # Recursos gráficos estáticos, sprites
```

```
|-- functions/           # Código Node.js para Firebase Functions
|-- scripts/             # Herramientas de automatización CLI
|-- AVISOS PARA EL ASESOR/ # Notas técnicas internas del equipo
`-- CARPETA INFORMATIVA/  # Documentación general y manuales
```